UNIT 3

Exercise 3 : Build a vector database using FAISS

Aim

To generate numerical embeddings for text and identify semantically similar sentences using embedding similarity.

Algorithm

- Start the program.
- Load a pretrained sentence-embedding model.
- Define a collection of text documents.
- Convert the documents into numerical vectors.
- Convert the user's query into an embedding.
- Calculate similarity scores between the query and documents.
- Display the most similar document.
- Stop.

Python Code and Output

```
import faiss

# Create an index for normalized vectors
dimension = embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)

index.add(embeddings.astype("float32"))

query = "Explain retrieval augmented generation"
query_vector = model.encode(
    [query], normalize_embeddings=True
).astype("float32")

scores, indices = index.search(query_vector, 3)

print("Top 3 retrieved documents:")

for rank, (score, idx) in enumerate(
    zip(scores[0], indices[0]), start=1
):
    print(rank, documents[idx])
    print("Similarity:", round(float(score), 4))
```

```
Top 3 retrieved documents:
1 Retrieval augmented generation uses documents to answer questions.
Similarity: 0.8485
2 Vector databases store embeddings for similarity-based retrieval.
Similarity: 0.411
3 Natural language processing helps computers understand human language.
Similarity: 0.3084
```

Result

Text embeddings were generated successfully, and semantic similarity search identified the most relevant document.